

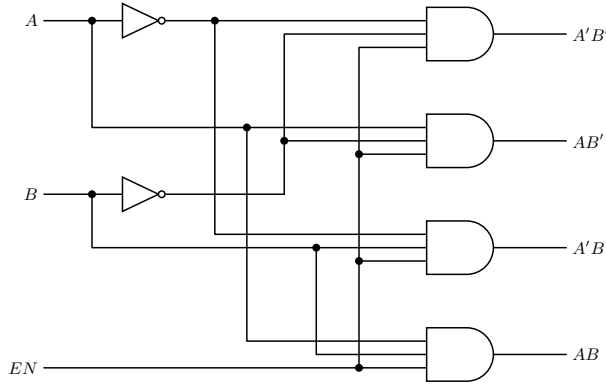
Homework 6

Week 6

6733172621 Patthadon Phengpinij

Collaborators. Claude (for L^AT_EX styling and grammar checking)

Problem 1. Design a Self-Checking 2:4 Demultiplexer with Enable.



E	A	B	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Solution

Number the outputs as in the truth table: $D_0 = EA'B'$, $D_1 = EA'B$, $D_2 = EAB'$, $D_3 = EAB$, i.e. the four AND gates of the figure, each gated by EN . Fault free, (D_0, D_1, D_2, D_3) carries exactly one 1 when $E = 1$ and is all-zero when $E = 0$. The first half of that is a 1-out-of-4 code, which is easy to check, but the idle word 0000 is not in it, so a checker watching the four outputs alone would report a fault every time the demux is disabled.

Step 1: make the output code constant weight. Add a fifth “idle” line $D_4 = E'$ (one inverter on EN). Then $(D_0, D_1, D_2, D_3, D_4)$ is a 1-out-of-5 codeword for *every* input combination, with $D_4 = 1$ marking the disabled state. Any fault that kills the active output, turns on a second one, or lets an output through while $E = 0$ now pushes the word off the code (weight 0 or 2).

Step 2: reduce 1-out-of-5 to two-rail signals. A pair of OR gates over a subset of the lines and its complement produces a two-rail (complementary) pair for every codeword. Such a pair misses a double-active word only when both active lines sit on the same side of the split, so with k pairs each line carries a k -bit membership signature and two lines are distinguished only if their signatures differ: $2^k \geq 5$, hence $k = 3$ pairs is the minimum. Splitting on “idle”, on A and on B gives three pairs, five OR gates in total:

$$P = (D_0 + D_1 + D_2 + D_3, D_4), \quad Q = (D_0 + D_1 + D_4, D_2 + D_3), \quad R = (D_0 + D_2 + D_4, D_1 + D_3).$$

Step 3: merge the pairs with two-rail checker cells. A TRC cell takes two two-rail pairs (a_0, a_1) , (b_0, b_1) and outputs

$$c_0 = a_0b_0 + a_1b_1, \quad c_1 = a_0b_1 + a_1b_0,$$

which is complementary iff both inputs are. Cascading two cells, $S = \text{TRC}(P, Q)$ and $Z = \text{TRC}(S, R)$, gives one final pair: $Z_0 \neq Z_1$ means healthy, $Z_0 = Z_1$ (00 or 11) means fault.

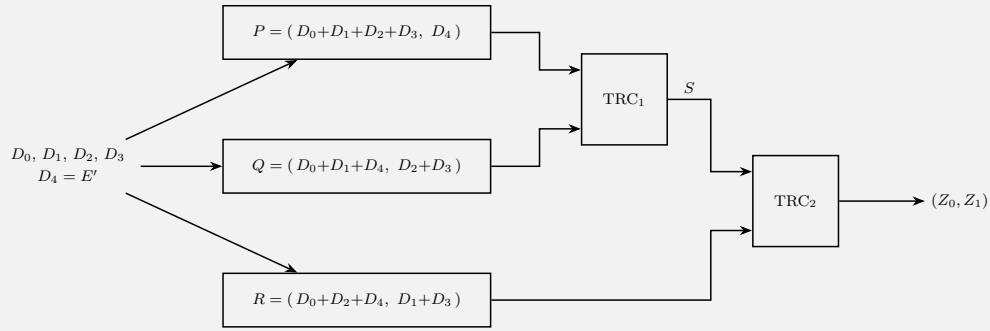


Figure 1: Checker for the demultiplexer: the 1-out-of-5 output word is compressed into three two-rail pairs and merged by two two-rail checker cells. $Z_0 \neq Z_1$ is fault free, $Z_0 = Z_1$ signals a fault.

Fault-free behaviour. All five normal words map to a complementary Z , and both values of Z occur:

E	A	B	Active line	P	Q	R	Z
0	X	X	D_4	01	10	10	01
1	0	0	D_0	10	10	10	10
1	0	1	D_1	10	10	01	01
1	1	0	D_2	10	01	10	01
1	1	1	D_3	10	01	01	10

Table 1: Checker response over the five normal input states (pairs written x_0x_1).

Coverage. Every single stuck-at fault inside the demux – on an inverter, on an AND gate input or output, or on an output line – either drops the active output, raises an extra one, or lets one through while $E = 0$. In all three cases the word has weight 0 or 2, at least one of P, Q, R becomes non-complementary, and the TRC cells propagate that to $Z_0 = Z_1$. For instance D_3 stuck-at-1 while D_0 is active gives $R = (1, 1)$, and the A branch stuck-at-0 with $E = 1$ silences every output and gives $P = (0, 0)$. The checker is itself self-testing: the five words above drive both rails of every OR group and both values of Z , so a single stuck-at fault inside the checker also shows up as a non-code Z for at least one normal input – the pair is totally self-checking. The one thing the output code cannot see is a fault on A or B *upstream* of the demux: the wrong output then fires alone, which is still a legal codeword, so catching that requires the select inputs to arrive in two-rail form and be checked separately.